

Javascript - Map, Filter e Reduce

Índice

Índice.....	1
Introdução.....	1
Map - Mapeamento.....	1
Filter - Filtrar.....	1
Reduce - Reduzir.....	2
Conclusão.....	2
Bibliografia.....	2

Introdução

Map, **reduce** e **filter** são todos métodos usados em arrays. Cada um destes métodos faz a iteração pelo array, e realiza uma transformação ou computação do array. Cada um destes métodos devolve um novo array baseado no resultado do método.

Todos estes métodos podem ser encadeados. Ex: `arr.filter(...).reduce(...)`

Map - Mapeamento

Map faz a iteração pelo array, e para cada valor do array vai fazer uma tarefa específica. Recebe uma função de callback e devolve um resultado (novo array).

É possível utilizar arrow functions para fazer o **Map**.

```
1  const arr = [10, 40, 20, 50, 80, 30];
2
3  const res = arr.map((val) => val + 10);
4  console.log(res);    // [20, 50, 30, 60, 90, 40]
```

Filter - Filtrar

Filter, como o nome diz, faz a filtragem dos elementos de um array que satisfaz uma condição. Tal como o Map, recebe uma função de callback, devolve um resultado, e pode usar arrow functions.

```
1  const arr = [10, 40, 20, 50, 80, 30];
2
3  const res = arr.filter((val) => val >= 50);
4  console.log(res);    // [50, 80]
```

Reduce - Reduzir

Reduce é usado quando existe a necessidade de reduzir um array num só elemento. Um caso prático é fazer a soma de todos os valores de um array.

Reduce tem dois parâmetros:

1. Função Callback (**callback(accumulator, currentValue)**). **Accumulator** é a variável que vai guardar o valor final. **currentValue** é uma variável que vai guardar o valor atual que vai ser iterado.
2. Valor inicial para o **accumulator**.

```
1  const arr = [10, 40, 20, 50, 80, 30];
2
3  const output = arr.reduce((accumulator, current) => {
4      accumulator = accumulator + current;
5      return accumulator;
6  }, 0)
7
8  console.log(output); // 230
```

Conclusão

A utilização do Map, Reduce e Filter é essencial para a simplificação e para a performance do código. Comparado com for loops em arrays não estáticos, **Map** é ~**1.63x** mais rápido, **Filter** é ~**1.11x** mais rápido, e **Reduce** é ~**1.05x** mais rápido. Contudo, encadear os 3 métodos é mais lento que um ciclo for, que é ~**2.37x** mais rápido. Mesmo para além da performance, é mais simples de se ler, e de se manter.

Bibliografia

<https://dev.to/localhostd3veloper/map-filter-and-reduce-in-javascript-51fj>

<https://www.skoumal.com/en/performance-of-built-in-higher-order-functions-map-filter-reduce-and-flatmap-vs-for-in-loop-in-swift/>